

# ParkVision 系统实现文档

## 1. 引言

### 1.1 编写目的

本文档是 ParkVision 项目在系统实现阶段形成的正式说明文档，主要用于对第五次答辩所展示的系统成果进行归纳、整理与书面化表达。与前期侧重项目立项、需求抽象和总体设计的文档不同，本阶段文档的核心任务已经从“说明我们准备做什么”转向“说明我们已经实现了什么、如何实现、实现到了什么程度”。

从答辩用途上看，本文档一方面服务于本次系统实现答辩，用于帮助评审教师快速理解系统已经完成的前端页面、后端结构、AI 功能接入和整体演示闭环；另一方面也服务于项目组后续的持续开发，为下一阶段的功能补全、系统联调、测试验证和部署优化提供阶段性基线。

因此，本文档的写作目标主要包括以下三个方面：

对系统实现阶段的实际完成内容进行客观梳理，明确本轮工作的完成边界；

对前端、后端、AI 以及业务闭环的落地情况进行结构化说明，体现工程实现过程；

在前四次答辩材料的基础上完成阶段衔接，使本次文档成为 ParkVision 项目开发链路中的自然延续。

### 1.2 前期工作基础与文档衔接

ParkVision 项目的推进并不是从本次系统实现阶段才开始，而是经历了从项目立项、项目管理、需求分析到系统设计的连续演进过程。本次第五次答辩所展示的实现成果，正是建立在前四次答辩文档逐步细化的基础之上。

第一次答辩阶段形成了《ParkVision 开题报告》。该阶段主要围绕项目背景、研究意义、总体愿景与核心创新点展开，明确了本项目面向立体智慧停车场景，重点解决高并发调度效率不足、复杂环境下安全防护薄弱以及传统停车系统交互僵化等问题，并提出了边缘计算、多智能体协同、Pre-Dispatch 预调度、AI 视觉感知和数字孪生监管等总体设想。

第二次答辩阶段形成了《ParkVision 项目管理文档》。该阶段进一步从项目范围、团队分工、进度安排、风险控制和质量保障等角度对项目实施过程进行了规划，回答了“项目如

何组织推进、如何分阶段落地、如何控制范围和风险”的问题，为后续开发提供了管理层面的执行框架。

第三次答辩阶段形成了《ParkVision 需求分析文档》。在这一阶段，项目组将前期较为宏观的构想收敛为可开发、可测试、可验证的需求规格，明确了系统边界、角色划分、关键业务场景以及功能性需求，建立了从车主端、管理端、监控端到调度中枢和 AI 服务的完整用例体系，使项目从概念层正式进入需求工程层。

第四次答辩阶段形成了《ParkVision 系统设计文档》。该阶段在需求基础上完成了总体架构、部署视图、逻辑架构、核心模块、数据模型和接口方案设计，进一步明确了云边端协同架构、后端分层方式、调度状态机、AI 接入链路以及前后端交互方式，为实现阶段的编码与联调提供了技术蓝图。

因此，本次《ParkVision 系统实现文档》并不是对前述文档内容的重复描述，而是在前四轮工作成果基础上的继续推进。它所关注的重点不再是“为什么要做”或“准备怎么做”，而是“前期分析与设计已经落地为哪些具体模块、页面、接口与演示能力”，从而体现项目从方案论证走向工程实现的阶段变化。

### 1.3 本轮答辩定位与实现范围

本次系统实现答辩是项目开发过程中的第五次阶段性答辩，其定位是对前期管理、需求和设计成果的工程化落地情况进行集中汇报。相较于前几轮更加偏向方案论证和设计推演的内容，本次答辩更强调可见、可用、可联调、可演示的系统成果。

结合当前项目进展，本轮实现范围主要集中在以下几个方面：

完成前后端分离的项目工程结构整理，形成较为清晰的开发目录和模块划分；

完成面向车主端、运营控制台、数字孪生大屏、调度中枢和 AI 感知页面的核心前端展示；

完成基于 Spring Boot 的后端分层结构，实现订单、车位、调度、告警、视觉接口等基础服务；

完成 AI 识别结果的结构化封装与业务链路接入，使 AI 能够参与订单创建、预调度说明和安全告警展示；

形成支撑答辩演示的最小系统闭环，使“入场识别、订单流转、库位变化、调度联动、告警反馈”能够在统一系统中得到展示。

需要说明的是，本轮答辩的重点在于“系统实现雏形”和“演示闭环”的建立，而不是一次

性完成全部真实生产能力的落地。因此，诸如 MySQL、Redis、RabbitMQ 的完整基础设施接入、真实边缘模型部署、硬件级实时通信以及更加复杂的工业级调度优化等内容，仍处于后续完善阶段。当前系统在若干模块中采用了 fallback 数据源、模拟识别结果和演示型策略逻辑，其目的并不是削弱系统能力，而是优先保证本轮答辩中的联调稳定性、结构完整性与展示连续性。

## 1.4 文档组织结构

为了与前四次文档的写作习惯保持一致，同时贴合本次系统实现阶段的汇报重点，本文档后续内容按“总览说明、模块实现、阶段总结”的思路组织，共分为六个部分：

第 1 章为引言，说明本文档的编写目的、与前期文档的衔接关系以及本轮答辩的定位；

第 2 章为项目实现总览，概括本轮实现目标、工程结构落地情况和核心演示闭环；

第 3 章为前端系统实现，重点说明车主端、控制台端、数字孪生与调度相关页面的实现情况；

第 4 章为后端微服务实现，重点说明后端分层结构、核心业务对象、接口组织方式以及回退数据机制；

第 5 章为 AI 功能实现，重点说明视觉识别、预测展示、智能交互及其与业务链路的联动方式；

第 6 章为项目实现总结，归纳本轮阶段成果、当前不足以及后续工作计划。

通过以上结构，本文档既能够与前期文档保持逻辑连续，又能够突出本次答辩真正需要回答的问题，即 ParkVision 项目在系统实现阶段已经完成了哪些关键落地工作，以及这些工作如何支撑项目后续继续向完整系统演进。

## 2. 项目实现总览

### 2.1 本轮实现目标

在完成前期项目管理、需求分析和系统设计之后，ParkVision 的开发工作进入了系统实现阶段。本轮工作的主要任务，已经从方案论证和设计推演，转向围绕既定技术路线完成可运行、可展示、可联调的系统雏形构建。换言之，本阶段不再单纯强调“系统应该具备哪些能力”，而是更关注“哪些能力已经在代码工程中完成落地，并能够支撑答辩中的完整展示”。

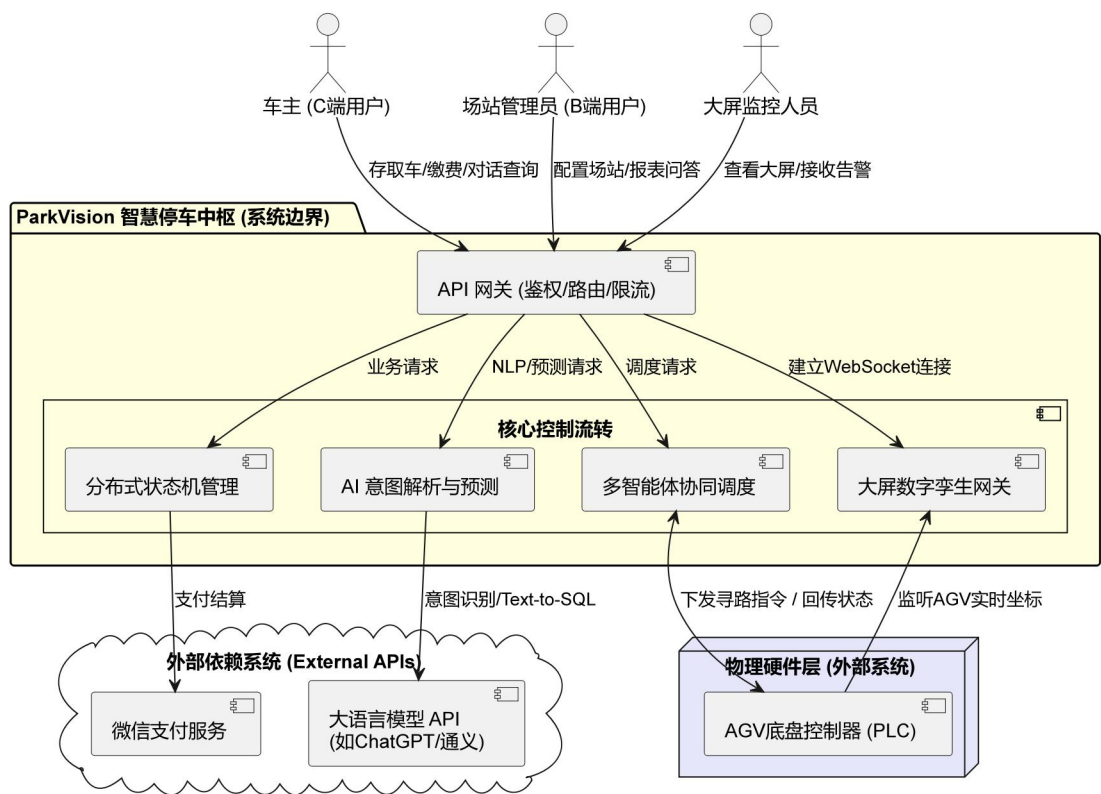
从阶段目标来看，本轮实现主要围绕三个层面展开。

第一，完成系统工程骨架的正式搭建。项目组对前后端代码、文档资料、启动脚本和部署草案进行了重新整理，使项目形成了较为清晰的工程目录结构。这一步的意义在于，ParkVision 不再停留于单个页面原型或零散功能演示，而是具备了继续进行模块化开发、联调测试和后续扩展的基本工程基础。

第二，完成主要业务界面的可视化实现。围绕开题与需求阶段确定的核心场景，项目已经实现了车主端、运营控制台、数字孪生大屏、AI 感知页和调度中枢等主要前端页面，能够从不同角色视角展示停车业务、调度行为和安全告警的运行状态。这使得系统实现成果不仅体现在代码层，也能够直观转化为答辩中的页面展示能力。

第三，完成后端业务结构和 AI 接入链路的最小闭环。当前后端已经采用 Spring Boot 构建了分层结构，具备订单、车位、调度、告警和视觉接口等基础服务；同时，AI 模块返回的结构化结果也已经能够进入后端业务逻辑，并在前端页面中体现为订单创建、预调度说明和安全急停等联动效果。这意味着本轮实现的重点，不是单点功能的堆砌，而是系统内部各模块之间的基础协同关系已经初步建立。

综合来看，本轮实现目标可以概括为“搭建工程骨架、完成主要页面、打通核心闭环、保障答辩展示”。这一定义既符合当前项目的开发实际，也与第五次答辩“系统实现”这一主题保持一致。



## 2.2 工程结构与技术栈落地

本轮实现阶段的一个重要成果，是项目工程结构已经由前期相对分散的资料与页面原型，整理为较为规范的前后端分离代码工程。当前项目根目录下主要包括 ``frontend``、``backend``、``docs``、``deploy``、``scripts``、``logs`` 以及一键启动脚本等内容，各目录职责边界较为明确，基本满足后续持续开发、答辩演示与工程维护的需要。

其中，``frontend`` 目录承载系统前端实现，采用 `Vue 3` 与 `Vite` 作为主要开发框架。当前前端已经实现总览驾驶舱、车主端主页、运营后台、数字孪生、AI 感知、调度中枢、动态定价、系统配置和室内外接续导航等多个页面，并通过统一状态管理方式组织页面间的数据联动。这样的结构安排，使前端不仅能够完成单页面展示，还能够支撑跨页面的业务流程串联。

``backend`` 目录承载系统后端实现，基于 `Spring Boot` 构建。后端代码已经按照 ``controller``、``service``、``repository``、``domain``、``dto``、``common``、``config`` 等层次完成组织。其中，控制层用于暴露 `RESTful API`，服务层负责核心业务逻辑，仓储层负责数据访问抽象，领域对象层负责承载停车业务中的订单、车位、AGV、告警和计费规则等核心实体。这种结构说明项目后端已经具备较清晰的分层设计基础，而不是简单的接口拼接。

``docs`` 目录用于沉淀项目全过程文档资料，既包括开题报告、项目管理文档、需求分析文档和系统设计文档，也包括本次系统实现答辩所依赖的 `PPT`、讲稿、架构说明和接口清单。该目录的存在，体现了项目组在开发过程中对文档连续性和过程留痕的重视，有利于答辩展示、团队协作和后续归档。

``deploy`` 与 ``scripts`` 目录则体现出项目的工程化意识。前者用于存放部署草案，后者用于本地启动和辅助处理，配合根目录中的 ``start.bat``、``start.ps1``、``stop.bat`` 和 ``stop.ps1``，使系统具备了更方便的演示启动方式。对于本次答辩而言，这些脚本和目录并非核心展示内容，但它们反映出项目已经从“做出页面”进一步转向“组织成工程”的实现状态。

从技术栈角度看，当前项目已经将前期设计文档中提出的部分技术路线落地为可运行代码。前端侧主要采用 `Vue 3`、`Vite` 和组件化页面组织方式实现多视图展示；后端侧采用 `Spring Boot` 与 `Java 17` 完成业务接口与模块分层；数据侧当前使用 `fallback` 数据源保障演示稳定，后续仍保留向 `MySQL`、`Redis`、`RabbitMQ` 等基础设施扩展的空间；AI 侧则以结构化 `JSON` 接口形式完成与业务系统的初步耦合。这种“先搭骨架、后补底座”的实现方式，较好地平衡了展示稳定性和后续可扩展性。

因此，可以认为本轮实现阶段已经完成了项目工程结构的正式收敛，并在技术栈层面将前期设计方案转化为初步可执行、可联调的系统实现基础。

## ▼ ParkVision-System

> backend

> deploy

> docs

> frontend

> logs

> scripts

🔒 .gitignore

📄 README.md

🪟 start.bat

➤ start.ps1

🪟 stop.bat

➤ stop.ps1

### 2.3 核心演示闭环概览

从系统实现答辩的展示逻辑来看，单独说明页面数量或接口数量并不足以体现项目完成度，真正能够反映实现水平的，是系统是否已经具备“从感知到业务再到可视化反馈”的闭环能力。基于当前实现成果，ParkVision 已经形成了一条较为完整的答辩演示主线。

该主线首先从模拟入场开始。系统通过 AI 识别流程输出车牌结果，并据此触发订单创建和车位分配。随后，前端总览页、订单区域和库位状态同步更新，使“识别结果进入业务系统”这一过程能够被直观观察。与前期仅停留在需求描述或架构图展示的状态相比，这一

步体现了前端页面、后端接口和数据状态之间已经建立起初步联动。

在完成入场后，系统进一步进入车主端业务环节。当前前端已经支持取车、临时取物、VIP 加急和支付等核心操作，这些操作虽然部分仍以演示型或模拟型方式实现，但它们已经能够在页面交互、订单状态变化和事件流更新之间形成对应关系。例如，临时取物能够体现挂起式业务逻辑，VIP 加急能够体现调度优先级变化，这些都使系统实现更贴近项目开题阶段提出的“柔性业务流转”目标。

在调度与监控层面，系统已经加入了高峰预测和 Pre-Dispatch 预调度展示逻辑。当预测到未来一段时间内取车压力上升时，系统可以生成预调度任务，并在调度队列、AGV 状态以及库位表现中反映“深层车辆迁移至缓冲区”的结果。与此同时，数字孪生页面能够以网格化方式展示库位变化和 AGV 集群状态，从而把后端状态变化转化为可视化反馈，体现“调度不是后台黑盒，而是前台可见”的特点。

在安全链路方面，系统还展示了人员闯入识别与急停反馈逻辑。当 AI 识别结果判定存在异常闯入时，系统能够切换到告警或急停状态，并在页面上展示相应的风险提示。这一过程虽然当前仍基于模拟推理结果，但已经表达出项目设计中的核心安全理念，即在智能停车场场景下，安全控制应始终高于一般业务执行逻辑。

因此，本次答辩中的核心演示闭环可以概括为：模拟入场识别、生成停车订单、更新车位状态、触发高峰预测、生成预调度任务、联动数字孪生展示，并在必要时通过 AI 异常识别进入安全告警链路。这条主线不仅覆盖了前端、后端和 AI 的主要实现成果，也能够较集中地体现 ParkVision 作为智慧停车系统的阶段性完整性。

## 3. 前端系统实现

### 3.1 车主端交互实现

在系统实现阶段，车主端的前端设计重点不再只是功能罗列，而是围绕“操作路径短、状态反馈直观、特色业务可感知”三个目标进行组织。当前项目中，车主端展示主要以高保真 H5 界面形式集成在统一前端工程中，通过模拟手机界面承载车辆状态查看、取车申请、临时取物、VIP 加急和支付等核心操作，从而在答辩演示中形成较强的代入感。

从页面表现上看，当前车主端已经能够展示用户所关心的核心停车信息，包括车辆所在位置、停放时长、当前电量、订单状态和可执行操作等内容。其中，“我的车辆”页面并非单纯的数据展示页面，而是被设计为车主与停车系统交互的主入口。用户在该页面中可以直接发起呼叫出库、临时取物和加急取车等行为，这种设计符合智慧停车场景下“低学习成本、少跳转操作”的交互需求。

从具体实现方式看，当前页面通过 Vue 3 的响应式状态组织交互逻辑。系统使用 `orderStatus`、`showOverlay`、`timer` 等状态变量表示车辆当前业务阶段，并通过 `doAction`

方法统一接收取车、取物、支付等操作请求。当用户触发临时取物时，页面会进入挂起式倒计时状态，展示取物剩余时间和“完成并入库”的操作按钮；当用户触发普通取车或支付时，页面状态则切换为对应的业务状态提示。这样的实现方式使得“Touch-and-Go 临时取物”这一前期设计中的特色功能，已经能够在页面层得到具体落地。

此外，车主端还通过明显的视觉层级强化了项目特色能力。例如，VIP 加急取车以高亮卡片形式独立展示，能够在界面上直观表现其“高优先级调度”的业务含义；临时取物功能则通过弹出式倒计时遮罩强调其时效性和连续计费特征；页面底部的功能说明区域还对实时车位、无感支付、AI 助手入口和加急调度等能力进行了归纳。这说明当前车主端实现不仅完成了页面搭建，也尝试把业务逻辑差异转化为用户可感知的界面语言。

综合来看，车主端前端实现已经基本覆盖了需求阶段提出的“可查询、可存取、可干预、可感知”的主要方向，并通过响应式状态控制把临时取物、VIP 加急等复杂业务流程转化为较为自然的交互体验。这为后续继续接入真实接口、支付链路和车主身份体系打下了良好的展示基础。



浙A·12345



## 我的车辆

存放中

📍 B2层 深库区 - A12位

停放时长  
**04:20**

当前电量  
**82%** ⚡



呼叫出库



免扣费

临时取物



**VIP 加急取车**

优先调度 AGV，预计省时 8 分钟

**+¥5.00**



车联



助手



我的

临时取物已到达交接区  
请在倒计时结束前完成取物



02:50

完成并入库

## 3.2 控制台端实现

与车主端强调轻量交互不同，控制台端的实现更强调信息汇聚、运营分析和管理可视化。当前前端工程中，控制台端已经围绕总览、报表、告警、计费规则和名单管理等内容构建了较为完整的运营视角页面，使系统不仅能够展示用户侧停车流程，也能够从经营管理侧体现平台化能力。

首先，在总览层面，系统已经实现驾驶舱式的首页视图。该页面通过指标卡片、流量预测图表、系统闭环链路说明、库区俯视图和事件流等模块，集中展示当前场站的占用率、车辆流量、AGV 在线状态和异常告警数量。这种布局方式使运营人员能够在单一页面中快速把握场站整体运行状态，也为答辩中的“系统已形成一体化展示界面”提供了最直观的表现。

其次，在管理功能层面，系统已实现面向运营人员的数据中台页面。当前 `AdminConsoleView` 通过标签页组织了 AI 智能报表、出入场记录、计费规则、异常告警和名单管理等功能区，并复用了统一的数据表格组件进行内容展示。这种方式一方面提高了页面结构的一致性，另一方面也使不同管理场景之间具备统一的交互风格，体现出系统前端在工程实现上的模块复用意识。

值得注意的是，控制台端当前已经加入了 AI 报表生成的展示能力。在智能报表页中，用户可以通过自然语言形式触发数据摘要生成，系统随后给出一段面向运营分析的文字结论，并配合 ECharts 图表展示收益趋势。这一部分虽然当前仍以演示型数据和模拟结果为主，但已经清晰体现出前期需求分析和系统设计中提出的 `Text-to-SQL` 与数据驱动运营思路，说明项目并未停留在传统后台表格展示，而是向智能化管理入口迈进。

此外，控制台端当前还承担了系统配置和接口管理入口的展示职责。虽然这部分功能尚未全部接入真实网关与设备层，但页面层已经预留出与边缘设备、AI 服务以及系统参数配置相关的管理位置，为后续接入真实环境提供了可扩展的前端入口。由此可见，本轮控制台端实现的价值不仅在于“看数据”，更在于初步构建了一个兼具监控、分析、治理和扩展能力的运营工作台。

总体而言，控制台端已经从简单的信息展示界面，发展为可承载管理视角、运营视角和分析视角的多功能前端模块，为 ParkVision 作为智慧停车综合管理平台的系统定位提供了重要支撑。



AGV 自动立体车库·云边协同调度

本地时间11:26:15 模拟入场 流量预测 全域急停

报表与记录

AI智能报表 入场记录 计费规则 异常告警 名单管理

Orders 管理 导出数据

实时同步底层业务流数据，支持高级筛选与导出。

| 订单号           | 车牌       | 事件         | 位置  | 状态   | 费用     |
|---------------|----------|------------|-----|------|--------|
| PV20260506001 | 沪A·P7686 | PARKED     | B18 | 停车中  | ¥18.00 |
| PV20260506002 | 沪D·E5218 | RETRIEVING | C05 | 取车中  | ¥42.50 |
| PV20260506003 | 苏E·M9021 | TOUCHING   | A09 | 临时取物 | ¥25.00 |
| PV20260506004 | 浙A·K1314 | FINISHED   | A01 | 已完成  | ¥16.00 |

### 3.3 数字孪生与调度页面实现

数字孪生与调度页面是本轮前端实现中最能体现项目特色的部分，也是系统从“信息记录平台”迈向“可视化调度平台”的关键体现。当前前端已经通过`TwinView`、`DispatchCenterView`以及首页事件流等模块，形成了对库位状态、AGV位置、任务队列和安全状态的联动展示。

在数字孪生页面中，系统将离散的车位对象和AGV坐标映射为可视化网格。当前实现使用计算属性将车位状态和AGV位置转换为二维格点，并根据空闲、占用、充电、缓冲和维护等状态赋予不同视觉表现。与此同时，AGV是否载车、当前位置以及执行任务也能够在页面侧栏中同步展示，使用户不仅能看到“库区状态”，还能够进一步理解“状态是如何由设备动作产生的”。

在安全展示方面，数字孪生页面还加入了急停联动逻辑。当系统判定存在闯入风险时，页面会切换至明显的告警视觉状态，显示 `EMERGENCY STOP` 标识，并同步展示边缘网关的安全提示信息。这种设计呼应了项目前期提出的“安全优先于一般业务逻辑”的原则，使数字孪生大屏不只是静态监控界面，而是承担起应急反馈窗口的职责。

与数字孪生相对应，调度中枢页面主要用于展示系统的任务组织能力。当前页面已经能够展示 AGV 调度队列、任务优先级、等待时间和策略规则等内容，并通过 VIP 加急、充电腾桩和 Pre-Dispatch 预调度等规则卡片，说明系统在调度层面并非简单采用线性 FIFO 队列，而是尝试引入多策略协同处理机制。虽然当前调度逻辑在实现上仍偏向演示型，但其界面结构已经较好地体现了项目设计中的“柔性调度”思想。

此外，总览页中的事件流展示也在前端联动中发挥了重要作用。无论是模拟入场、AI 识别完成、预调度触发、VIP 插队还是安全急停，系统都会将关键变化写入事件流模块，从而让评审者能够以时间顺序感知系统内部状态的变化过程。这种设计使多个页面之间形成了较自然的叙事链条，增强了答辩时的可讲述性和系统运行感。

因此，数字孪生与调度页面的实现，不仅补齐了可视化展示层，也把车位、AGV、任务和安全这四类核心对象组织到了统一的前端语境中。它们与车主端和控制台端共同构成了本轮系统实现阶段前端部分的主要成果。



## 4. 后端微服务实现

### 4.1 分层架构实现

在系统实现阶段，后端部分的核心目标并不是一次性完成全部基础设施与复杂分布式能力的落地，而是先建立一套结构清晰、职责明确、便于扩展的业务服务骨架。基于这一目标，当前 ParkVision 后端采用 Spring Boot 构建，并按照 `controller`、`service`、`repository`、`domain`、`dto`、`common` 和 `config` 等层次组织代码，形成了较为标准的分层架构。

从控制层来看，当前系统已经围绕停车业务、调度逻辑和 AI 感知三个方向提供了基础 RESTful 接口。例如，`OrderController` 负责接收模拟入场、取车、临时取物和支付等请求；`DispatchController` 负责提供调度队列、AGV 状态、预调度和 VIP 加急接口；`VisionController` 则负责接收视觉推理请求并返回结构化识别结果。这样的接口组织方式说明后端已经开始按照业务域进行控制层划分，而不是将所有功能堆叠在单一入口中。

在服务层中，系统进一步把“请求接收”与“业务处理”解耦。以订单模块为例，控制层只负责解析路径参数和调用服务方法，而真正的订单创建、状态切换、车位占用更新和金额处理逻辑，则由 `OrderService` 完成；同样，预调度任务生成、AGV 状态调整和 VIP 插队逻辑由 `DispatchService` 负责；视觉识别结果的封装与动作判断，则由 `VisionService` 负责。这一分层方式使业务逻辑不直接暴露在接口层，有利于后续继续增加校验规则、替换数据源或扩展调度策略。

仓储层在当前系统中承担的是“数据访问抽象”而不是“数据库实现细节”本身。通过 `ParkVisionRepository` 接口，服务层只依赖“获取车位”“查询订单”“新增调度任务”“查询告警”等抽象能力，而不关心数据来自真实数据库、缓存中间件还是内存模拟数据。这一点对于当前系统实现阶段尤为关键，因为它保证了后端业务骨架可以先稳定落地，再在后续逐步接入 MySQL、Redis 或消息队列等真实基础设施。

`domain` 层用于表达停车业务中的核心对象，例如订单、车位、AGV、调度任务、告警事件和计费规则等。这些对象使业务表达更加清晰，也为前端页面、接口返回和后续数据库建模提供了统一语义。`dto` 层则承担前后端传输结构的职责，例如视觉请求与视觉返回结果都通过专门的数据传输对象定义，从而避免前端直接依赖内部领域模型。

此外，系统还通过 `common` 层提供统一返回格式和异常处理机制。当前所有接口均通过 `ApiResponse` 返回标准化数据结构，包含成功状态、编码、消息、数据体和时间戳；业务异常通过 `BusinessException` 抛出，再由 `GlobalExceptionHandler` 统一转换为可前端识别的失败响应。这说明当前后端虽然规模不大，但已经具备了较好的工程规范意识，而不是仅完成“接口能跑通”的最低实现。

总体而言，本轮系统实现阶段已经在后端完成了从“页面配套接口”向“可扩展业务服务结构”的重要过渡。虽然真实数据库、中间件和复杂并发控制仍有待继续增强，但后端分层

架构本身已经具备较清晰的工程基础，这也是本次答辩能够稳定展示业务闭环的重要前提。

```

  ▾ backend
    ▾ src
      ▾ main
        ▾ java\com\parkvision\cps
          > common
          > config
          ▾ controller
            J AdminController.java
            J DashboardController.java
            J DispatchController.java
            J OrderController.java
            J ParkingController.java
            J VisionController.java
          ▾ domain
            > admin
            > dashboard
            > dispatch
            > order
            > parking
          ▾ dto
            > parking
            > vision
          ▾ repository
            J FallbackParkVisionRepository.java
            J ParkVisionRepository.java
          ▾ service
            J AdminService.java
            J DashboardService.java
            J DispatchService.java
            J OrderService.java
            J ParkingService.java
            J VisionService.java
            J ParkVisionApplication.java
          ▾ resources

```

```
1 package com.parkvision.cps.controller;
2
3 import com.parkvision.cps.common.ApiResponse;
4 import com.parkvision.cps.domain.order.OrderStatus;
5 import com.parkvision.cps.domain.order.ParkingOrder;
6 import com.parkvision.cps.service.OrderService;
7 import org.springframework.web.bind.annotation.PathVariable;
8 import org.springframework.web.bind.annotation.PostMapping;
9 import org.springframework.web.bind.annotation.RequestMapping;
10 import org.springframework.web.bind.annotation.RestController;
11
12 @RestController
13 @RequestMapping("/api/orders")
14 public class OrderController {
15     private final OrderService orderService;
16
17     public OrderController(OrderService orderService) {
18         this.orderService = orderService;
19     }
20
21     @PostMapping("/entry")
22     public ApiResponse<ParkingOrder> simulateEntry() {
23         return ApiResponse.created(orderService.simulateEntry());
24     }
25
26     @PostMapping("/{orderNo}/retrieve")
27     public ApiResponse<ParkingOrder> retrieve(@PathVariable String orderNo) {
28         return ApiResponse.ok(orderService.changeStatus(orderNo, OrderStatus.RETRIEVING));
29     }
30
31     @PostMapping("/{orderNo}/touch-and-go")
32     public ApiResponse<ParkingOrder> touchAndGo(@PathVariable String orderNo) {
33         return ApiResponse.ok(orderService.changeStatus(orderNo, OrderStatus.TOUCHING));
34     }
35
36     @PostMapping("/{orderNo}/pay")
37     public ApiResponse<ParkingOrder> pay(@PathVariable String orderNo) {
38         return ApiResponse.ok(orderService.changeStatus(orderNo, OrderStatus.FINISHED));
39     }
40 }
41
```



You, 44分钟前 | 1 author (You)

```
16 @Service
17 public class OrderService {
18     private static final List<String> PLATES = List.of("沪A·P7686", "沪D·E5218", "苏E·M9021", "沪
19
20     private final ParkVisionRepository repository;
21     private final Random random = new Random();
22
23     public OrderService(ParkVisionRepository repository) {
24         this.repository = repository;
25     }
26
27     public ParkingOrder simulateEntry() {
28         ParkingSlot slot = repository.findFirstAvailableSlot()
29             .orElseThrow(() -> new BusinessException(code: "NO_AVAILABLE_SLOT", message: "当
30         String plate = PLATES.get(random.nextInt(PLATES.size()));
31         slot.setStatus(plate.contains("D") ? SlotStatus.CHARGING : SlotStatus.OCCUPIED);
32
33         ParkingOrder order = new ParkingOrder(
34             "PV" + String.valueOf(System.currentTimeMillis()).substring(5),
35             plate,
36             slot.getId(),
37             LocalDateTime.now(),
38             OrderStatus.PARKED,
39             BigDecimal.ZERO
40         );
41         return repository.saveOrder(order);
42     }
43
44     public ParkingOrder changeStatus(String orderNo, OrderStatus status) {
45         ParkingOrder order = repository.findOrderByNo(orderNo)
46             .orElseThrow(() -> new BusinessException(code: "ORDER_NOT_FOUND", message: "订
47         order.setStatus(status);
48         if (status == OrderStatus.FINISHED) {
49             order.setAmount(new BigDecimal("18.00"));
50         }
51         return order;
52     }
53 }
54
```

## 4.2 核心业务对象与状态流转

从当前后端代码结构来看，ParkVision 的业务组织已经围绕停车场景中的关键实体展开，其中最核心的对象包括停车订单、物理车位、AGV 单元、调度任务以及告警与计费相关对象。它们共同构成了系统当前的最小业务模型，也是前端页面和演示闭环能够成立的基础。

首先，停车订单 `ParkingOrder` 是当前后端业务中的核心主线对象。每一笔订单都记录了订单编号、车牌号、关联车位、入场时间、订单状态和费用金额等信息。之所以将订单作为系统主线，是因为它天然连接了车主行为、车位状态、调度动作和结算流程。在目前的实现中，车辆模拟入场后会直接生成订单，并以 `PARKED` 作为初始状态；后续用户发起取车、临时取物或支付等动作时，订单状态随之变化，从而把业务流程统一串联在订单生命周期之上。

与订单相对应，`ParkingSlot` 用于描述物理车位的基本状态。当前系统为车位定义了编号、所在层级和状态等字段，并通过 `SlotStatus` 枚举对状态进行约束。现阶段支持的车位状态包括 `EMPTY`、`OCCUPIED`、`CHARGING`、`BUFFER` 和 `MAINTENANCE`，分别对应空

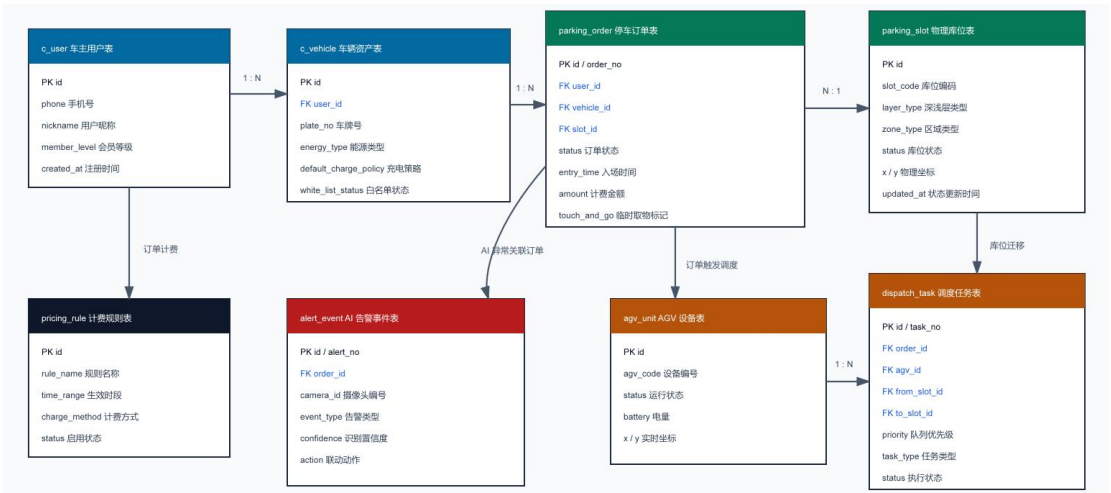
闲、占用、充电、缓冲和维护等典型场景。这样的状态定义一方面便于后端在分配车位和生成调度任务时进行逻辑判断，另一方面也使前端可以根据统一枚举渲染颜色和标签，保证前后端语义一致。

在调度相关对象中，`AgvUnit` 用于表示执行搬运任务的 AGV 设备，当前包含设备编号、坐标位置、是否载车和当前任务描述等信息；`DispatchTask` 则用于表示调度队列中的一项任务，包含关联车牌、任务类型、标签、预计等待时间和优先级标记。这两个对象共同承担了“任务谁来执行、当前执行到什么状态”的表达职责，使调度行为能够从后端对象层自然传递到前端调度中枢和数字孪生页面。

在状态流转方面，当前后端已经具备较明确的订单状态机雏形。订单在模拟入场后进入 `PARKED` 状态，表示车辆已完成入库；用户发起取车时，订单切换到 `RETRIEVING`，表示进入出库调度阶段；用户临时取物时，订单切换到 `TOUCHING`，用于表示挂起式取物流程；支付完成后，订单进入 `FINISHED` 状态，表示本次停车流程结束。虽然 `PAYING` 和 `ABNORMAL` 等状态当前尚未在完整流程中全部展开，但状态枚举已经为后续补充支付中间态和异常流程预留了空间。

需要强调的是，这种状态流转虽然当前仍以演示型实现为主，但其意义在于建立了统一的业务语言。后续无论是接入真实计费引擎、加入更复杂的调度回滚，还是引入消息队列与设备回执，都可以围绕现有状态对象继续拓展，而不必推翻当前的整体模型。这也是系统设计在实现阶段得以平滑落地的重要原因。

因此，可以认为当前后端已经具备一套以停车订单为主线、以车位和调度对象为支撑、以状态流转为核心约束的业务表达方式。这套模型虽然尚未达到完整工业系统的复杂程度，但已经足以支撑本轮答辩中的核心业务演示。



```

3 public class ParkingSlot {
4     private final String id;
5     private final String layer;
6     private SlotStatus status;
7
8     public ParkingSlot(String id, String layer, SlotStatus status) {
9         this.id = id;
10        this.layer = layer;
11        this.status = status;
12    }
13

```

```

3 public enum SlotStatus {
4     EMPTY,
5     OCCUPIED,
6     CHARGING,
7     BUFFER,
8     MAINTENANCE
9 }

```

>

```

44 public ParkingOrder changeStatus(String orderNo, OrderStatus status) {
45     ParkingOrder order = repository.findOrderByNo(orderNo)
46         .orElseThrow(() -> new BusinessException(code: "ORDER_NOT_FOUND", "订单不存在: "
47         order.setStatus(status);
48     if (status == OrderStatus.FINISHED) {
49         order.setAmount(new BigDecimal("18.00"));
50     }
51     return order;
52 }
53 }

```

### 4.3 接口设计与回退数据机制

在当前系统实现阶段，后端接口设计和回退数据机制是保证联调稳定性的重要组成部分。对于答辩系统而言，如果一开始就强依赖完整数据库、缓存和外部 AI 服务，虽然技术上更“真实”，但也会显著增加联调成本和演示不确定性。因此，当前后端采取了“接口先成型、数据先稳定、底层逐步替换”的实现策略。

从接口设计上看，系统已经围绕总览、车位、订单、调度、管理后台和 AI 感知等业务域提供了较完整的基础 API。订单域支持模拟入场、取车、临时取物和支付；调度域支持查询任务队列、查询 AGV 状态、触发预调度和插入 VIP 任务；AI 域支持统一的视觉推理接口；首页与后台域则支持获取汇总指标、预测数据、订单记录、告警信息和计费规则等内容。这说明当前接口设计已经不是“为单页面临时写接口”，而是开始具备面向完整业务域的组织意识。

在接口返回形式上，系统统一使用 `ApiResponse` 作为标准响应结构，对成功和失败场景进行封装。这种统一返回方式一方面便于前端在不同页面复用请求处理逻辑，另一方面也使接口更具规范性，有利于后续补全文档、进行测试或继续对接其他模块。同时，`BusinessException` 与 `GlobalExceptionHandler` 的组合，也使当前系统能够在车位不足、订单不存在或请求非法等场景下返回结构化错误信息，而不是直接暴露内部异常。

与接口设计相匹配的，是当前后端使用的 `fallback` 数据机制。系统通过 `ParkVisionRepository` 定义统一的数据访问抽象，并由 `FallbackParkVisionRepository` 提供默认实现。该实现通过内存中的集合预置了车位、订单、AGV、调度队列、告警和计费规则等数据，并对外提供“查询首个空闲车位”“查询深层占用车位”“查找订单”“插入调度任务”等方法。与此同时，类上的条件注解也使其能够在当前演示模式下作为默认数据源启用。

这种回退数据机制在本轮实现阶段具有三方面作用。第一，它保证了前后端联调过程的稳定性，即使数据库尚未正式接入，前端页面和后端接口仍然能够返回完整数据；第二，它使核心业务链路可以被可靠演示，例如模拟入场后生成订单、预调度后缓冲区状态变化、VIP 插队后任务进入队首等；第三，它为后续替换真实数据源保留了清晰边界，未来只需新增基于 MySQL、Redis 或消息队列的仓储实现，而不必大幅修改服务层逻辑。

当然，需要客观指出的是，`fallback` 数据源本质上仍是系统实现阶段的演示型支撑能力，并不等同于真实部署环境中的完整持久化方案。但从本轮答辩目标来看，这种设计是合理的。它使当前后端能够优先完成“接口稳定、结构清晰、展示连续”的任务，同时也为后续真实化接入打下了较整洁的抽象基础。

综上所述，当前后端通过统一接口组织与回退数据机制，较好地平衡了工程规范、联调效率与答辩稳定性，使系统能够在基础设施尚未完全落地的情况下，依然形成可演示、可扩展、可继续演进的后端实现框架。

```

13 public interface ParkVisionRepository {
14     List<ParkingSlot> findSlots();
15
16     Optional<ParkingSlot> findFirstAvailableSlot();
17
18     Optional<ParkingSlot> findFirstDeepOccupiedSlot();
19
20     List<ParkingOrder> findOrders();
21
22     Optional<ParkingOrder> findOrderByNo(String orderNo);
23
24     ParkingOrder saveOrder(ParkingOrder order);
25
26     List<AlertEvent> findAlerts();
27
28     List<PricingRule> findPricingRules();
29
30     List<AgvUnit> findAgvUnits();
31
32     List<DispatchTask> findDispatchQueue();
33
34     DispatchTask enqueueDispatchTask(DispatchTask task);
35 }

```

## 5. AI 功能实现

### 5.1 视觉识别接口实现

AI 功能是 ParkVision 项目在本轮系统实现阶段中最具辨识度的组成部分之一。与传统停车系统只把识别结果作为辅助信息不同，本项目希望让 AI 从“展示层能力”进一步进入“业务决策链路”。因此，在当前系统实现中，AI 部分的重点并不只是页面上出现识别框和模型名称，而是已经围绕统一输入、统一输出和统一动作语义构建了基础接口形式。

从接口设计来看，当前后端通过 `VisionController` 对外提供 `/api/edge/vision/infer` 接口，用于接收视觉推理请求。请求对象 `VisionRequest` 当前包含 `cameraId`、`imageUrl` 和 `simulateIntrusion` 三个核心字段，分别用于表示采集来源、图像输入位置以及是否模拟异常闯入场景。这种设计方式使视觉服务在逻辑上已经具备了“边缘节点输入 -> 后端处理 -> 结构化结果返回”的基本输入规范。

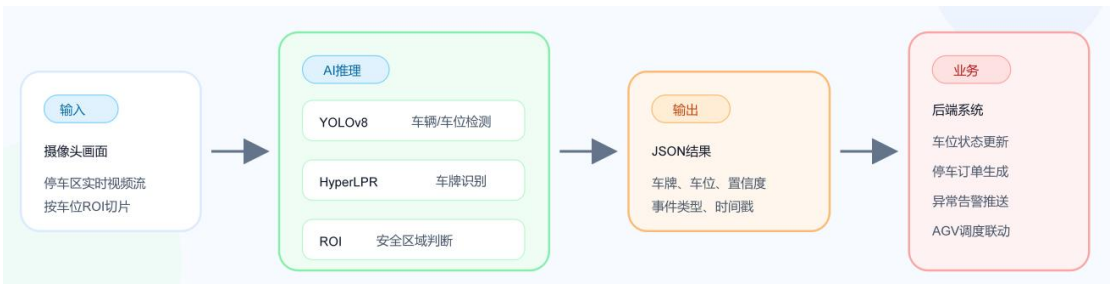
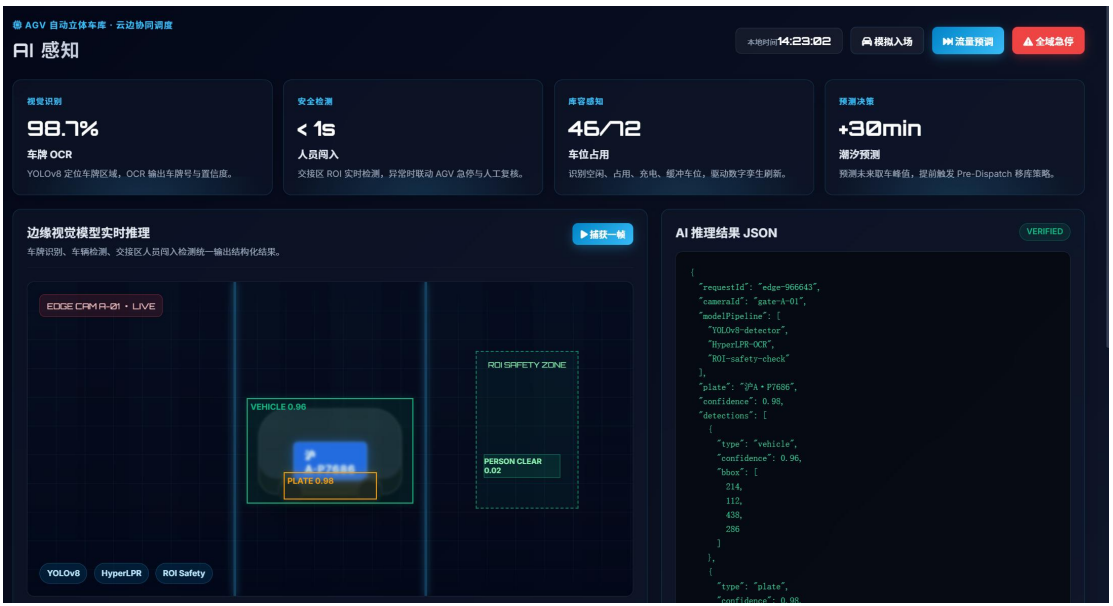
在返回结构上，系统使用 `VisionResult` 统一定义 AI 推理结果，当前包括 `requestId`、`cameraId`、`plate`、`confidence`、`intrusion` 和 `action` 等字段。其中，`plate` 和 `confidence` 用于表达车牌识别结果及其置信度，`intrusion` 用于表示是否识别到交接区异常闯入，`action` 则用于输出后续建议执行的业务动作。这种结构的意义在于，AI 输出不再是难以直



接处理的原始视觉结果，而是已经被规整为后端业务系统可以直接消费的标准化学段集合。

在当前实现中，`VisionService` 通过模拟方式生成车牌号、置信度和动作判断结果。当未检测到闯入风险时，系统默认返回 `ALLOW\_ENTRY\_AND\_CREATE\_ORDER`，表示允许入场并触发订单创建；当存在异常闯入时，则返回 `ESTOP\_AND\_REVIEW`，表示进入急停和人工复核流程。虽然这一逻辑目前仍属于演示型实现，但它已经清晰地体现出 AI 结果不再仅停留在“识别到了什么”，而是进一步回答“系统接下来应该做什么”。

因此，本轮 AI 视觉接口实现的价值，主要体现在三个方面：一是完成了输入输出格式的统一；二是形成了从识别字段到业务动作的映射；三是为未来接入真实边缘模型服务保留了稳定的接口边界。也就是说，即便当前阶段尚未引入完整的真实推理服务，AI 功能的接口形态和语义约束已经基本确定，为后续继续替换底层实现创造了良好条件。



## 5.2 预测与智能交互展示

除了视觉识别接口之外，当前系统在前端层面还完成了潮汐预测、智能助手和统一 AI 页面展示逻辑的初步实现。这说明本轮 AI 功能并没有被局限于单一的摄像头识别场景，而是已经开始向“感知 + 预测 + 交互”的综合能力方向延伸。

在预测展示方面，当前 `AiVisionView` 页面通过 `forecastBars` 计算属性对未来一段时间内的出库压力进行可视化表达，并将不同风险等级划分为低、中、高三类，从而让预测结果不再只是数字序列，而是能够通过柱状结构和颜色层级直观表现压力变化趋势。页面同时给出“建议提前迁移车辆数量”和“平均等待时间预计下降比例”等解释性文本，使预测结果能够自然过渡到调度建议，这种做法较好地体现了前期设计中“预测为业务服务”的思路。

在智能交互方面，当前系统已经通过演示型方式展示自然语言意图识别链路。AI 页面中的“智能助手意图识别”模块，以“用户意图 - 意图识别 - 工具调用 - 系统反馈”的流程呈现了从一句自然语言请求到具体业务动作的转化过程。例如，用户提出“我要现在取车，能不能快一点？”，系统将其解释为 `retrieve\_car + vip\_priority`，进一步映射为调用 `POST /api/dispatch/vip` 接口，并返回高优先级队列已插入的业务反馈。这一设计虽然目前主要用于展示，但已经把前期需求中的自然语言交互目标具体化为可理解的系统处理过程。

此外，当前 AI 页面还通过顶部能力卡片的形式，对视觉识别、安全检测、库容感知和预测决策四类能力进行了分项展示。这样的实现方式有两个好处：一方面它为答辩现场提供了高度概括的能力总览，另一方面也使页面从一开始就明确传达出“AI 能力是多模块协同的，而不是单点识别模型”的项目定位。尤其是在视觉识别与预测决策并列呈现的情况下，系统能够更好地表现“AI 不只看当前，还参与预判未来”的设计思想。

总体来看，本轮在预测和智能交互方面的实现，虽然仍以演示型数据和固定流程为主，但已经成功把“AI 预测高峰”“AI 理解用户意图”“AI 输出解释性结论”这些抽象概念转化为前端页面中可以直接展示的功能模块。这使得 AI 功能在答辩中不再是独立一页的概念介绍，而是能够真实嵌入系统整体演示流程中的组成部分。



```
1 package com.parkvision.cps.dto.vision;
2
3 You, 15小时前 | 1 author (You)
4 public record VisionResult(
5     String requestId,
6     String cameraId,
7     String plate,
8     double confidence,
9     boolean intrusion,
10    String action
11 ) {
12 }
```

### 5.3 AI 与业务链路联动

从系统实现视角看，AI 功能真正体现价值的关键，不在于页面上显示了多少模型名称，也不在于是否单独构造了一段识别 JSON，而在于这些结果是否已经能够进入系统业务链路。当前 ParkVision 在这方面已经完成了一个较为明确的雏形，即 AI 识别和预测结果已经开始与订单创建、调度行为和安全告警发生联动。

在入场链路中，AI 识别的输出结果可以作为订单生成的触发依据。当前后端的视觉结果中，若 `action` 返回为允许入场，则系统可以进一步进入订单创建和车位分配流程；这意味着 AI 输出已经承担了业务起点的判断作用，而不只是向页面返回一个“识别通过”的标签。结合前端的总览页和事件流展示，这种联动能够形成较清晰的答辩叙事：识别车牌、确认动作、生成订单、更新车位。

在调度链路中，AI 的作用则主要体现在预测与优先级辅助决策上。当前系统虽然尚未部署真实时序预测模型，但已经在页面和业务逻辑中引入了 Pre-Dispatch 预调度展示语义，即当未来出库压力升高时，系统给出提前移库建议，并在调度队列与库位状态中体现出“深层车辆迁移到缓冲区”的结果。这种设计说明 AI 已经开始被视为调度规则的前置输入，而不是调度完成后的补充说明。

在安全链路中，AI 识别结果与急停机制之间的关系更加直接。当前当 `intrusion` 被判定为真时，系统会将动作切换到急停或人工复核语义，前端页面也会同步切换到风险展示状态。这说明项目已经明确将 AI 的异常检测能力与安全控制链路相连，使“识别闯入”能够对应“暂停调度”这一更高优先级的业务响应。这种联动方式十分契合智慧停车场景的实际需求，



也强化了本项目与一般停车管理系统之间的差异性。

更进一步看，当前 AI 页面中用于展示的“云边协同 AI 链路”已经把这套关系抽象为较清晰的五步流程，即摄像头采集、边缘 AI 推理、结构化 JSON、Java 业务决策和 AGV 调度联动。虽然现阶段底层模型与硬件链路仍未全部真实接入，但这条链路已经把 AI 和业务之间的耦合边界表达得比较明确：AI 负责感知和结构化判断，后端负责业务规则和流程控制，而前端负责把联动结果展示出来。这样的职责分工为后续继续完善真实模型服务、硬件网关和数据闭环奠定了清晰的系统边界。

因此，可以认为本轮 AI 功能实现的核心价值并不是“模型已经完全训练完成”，而是“AI 结果已经开始真正进入停车业务系统内部”。这一步对于系统实现答辩而言非常关键，因为它证明了 ParkVision 的 AI 部分并不是装饰性模块，而是正在逐步成为订单、调度和安全控制三条主链路中的有效输入。

```
12 @RestController
13 @RequestMapping("/api/edge/vision")
14 public class VisionController {
15     private final VisionService visionService;
16
17     public VisionController(VisionService visionService) {
18         this.visionService = visionService;
19     }
20
21     @PostMapping("/infer")
22     public ApiResponse<VisionResult> infer(@RequestBody(required = false) VisionRequest request) {
23         return ApiResponse.ok(visionService.infer(request));
24     }
25 }
26
```

## 6. 项目实施总结

### 6.1 阶段成果

经过本轮系统实现阶段的开发与联调，ParkVision 已经从前期的方案设计和需求论证，推进到具备实际页面展示、后端接口支撑、AI 结果接入和业务闭环演示能力的系统雏形。从第五次答辩的目标来看，本阶段最重要的成果，不是单一模块数量的增加，而是项目已经形成了一套能够被“看见、讲清和继续扩展”的整体实现框架。

首先，在工程组织层面，项目已经完成从分散式原型到前后端分离工程的结构收敛。当前系统具备独立的前端工程、后端工程、文档资料、部署草案和启动脚本，说明项目不再只是若干界面和示意图的集合，而是已经具备继续进行模块化开发、系统联调和演示部署的基础条件。这一成果为后续项目迭代提供了稳定的工程骨架。

其次，在前端实现层面，系统已经完成了面向不同角色的核心页面落地。车主端实现了车辆状态查看、取车、临时取物、VIP 加急和支付等主要交互；控制台端实现了总览驾驶舱、

报表与记录、告警与规则展示等管理视图；数字孪生与调度页面则进一步把车位、AGV、任务队列和安全状态组织到统一的可视化场景中。由此，前端已经能够较完整地承载本轮答辩所需要的展示主线。

再次，在后端实现层面，系统已经形成了较规范的分层结构，并完成订单、车位、调度、告警和视觉接口等基础服务。虽然当前底层数据仍以 `fallback` 机制为主，但后端已经具备清晰的控制层、服务层、仓储层和领域对象划分，且统一响应与异常处理机制也已具备雏形。这意味着当前系统不仅“能演示”，而且在代码组织上已经为后续真实化接入留下了较好的扩展空间。

最后，在 AI 功能层面，系统已经完成了从视觉识别结果到业务联动的初步打通。当前 AI 部分不仅展示了车牌识别、闯入检测、潮汐预测和智能助手等能力，还通过结构化 JSON 和动作字段把识别结果接入订单创建、调度建议和安全急停链路。这一点使 ParkVision 的 AI 模块区别于仅用于展示的视觉 Demo，而开始体现出其作为业务输入的实际价值。

综合来看，本轮系统实现阶段已经完成了“工程结构搭建、主要页面落地、后端骨架成型、AI 接口接入、核心演示闭环形成”五个方面的核心任务。这些成果已经足以支撑系统实现答辩，也为项目继续向真实数据、真实设备和真实模型服务演进奠定了基础。

## 6.2 当前不足

尽管本轮系统实现已经取得了较为完整的阶段成果，但从严格的软件工程标准和真实场景落地要求来看，当前系统仍然存在一些明显的不足。这些不足并不意味着本轮工作价值不足，而是表明项目仍处于“系统实现雏形”阶段，尚未完全进入生产级部署与全链路验证阶段。

首先，数据层和基础设施尚未完全真实化。当前后端虽然已经完成仓储抽象和接口组织，但主要仍依赖 `FallbackParkVisionRepository` 提供内存级演示数据，尚未正式接入 MySQL、Redis、RabbitMQ 等基础设施。因此，现阶段系统在数据持久化、并发一致性、异步任务处理和缓存优化等方面，仍未完全体现真实生产环境中的复杂性。

其次，AI 推理链路当前仍以模拟结果和展示型逻辑为主。虽然系统已经完成了视觉接口定义、结构化返回和业务动作映射，但底层并未真正接入完整的边缘模型推理服务，部分页面中的识别结果、预测结果和意图识别链路仍主要服务于答辩演示。也就是说，AI 的业务语义已经确定，但模型服务本身的精度、时延和稳定性还有待后续通过真实联调进一步验证。

再次，调度逻辑当前仍偏向演示级实现。系统已经能够表现 VIP 插队、Pre-Dispatch 预调度和临时取物等特色业务，但当前调度任务生成、优先级处理和 AGV 状态变化仍以模拟和规则化表达为主，尚未达到真实工业停车系统中对路径冲突控制、设备反馈闭环、任务回滚处理和多车协同调度的完整深度。

此外，实时通信与设备联动能力仍有待增强。当前数字孪生页面和 AGV 状态更多依赖

前端状态更新和演示数据驱动，尚未全面接入 **WebSocket** 推送、硬件心跳、**PLC** 网关反馈等真实实时链路。这样虽然有利于本轮答辩的稳定展示，但也意味着系统在实际连续运行场景下的实时性和鲁棒性还缺少足够验证。

最后，系统测试和部署验证还不够充分。当前项目已经具备较好的展示基础，但高并发压力测试、异常恢复测试、安全边界测试和部署环境一致性验证仍未形成完整闭环。这使得系统目前更适合作为“阶段性实现成果”进行汇报，而不是直接视作可部署到真实智慧停车场站的成熟产品。

因此，对当前系统的合理评价应当是：它已经完成了从概念方案向工程雏形的关键跨越，但距离完整的、可生产落地的智慧停车系统仍有若干重要环节需要继续补强。

## 6.3 后续工作计划

围绕当前系统存在的不足，**ParkVision** 在下一阶段的推进重点将从“做出雏形、支撑答辩”逐步转向“补齐底座、增强联调、提高真实度”。后续工作计划可以从数据层、**AI** 接入、调度能力、实时通信以及测试部署五个方面展开。

第一，进一步完善数据层与基础设施接入。后续需要逐步将当前 **fallback** 数据机制替换为真实持久化方案，完成 **MySQL** 表结构落地、**Redis** 状态缓存设计以及消息中间件的接入，从而使订单、车位、调度和告警等核心数据具备更真实的存储与流转环境。这一步将是系统从演示型后端过渡到真实业务后端的关键。

第二，推进真实 **AI** 服务联调。后续应将当前视觉接口与具体模型服务正式衔接，包括车牌识别、人员闯入检测、车位占用感知和时序预测等能力，并围绕识别精度、响应速度和异常稳定性开展验证。与此同时，智能助手和自然语言报表功能也可以逐步从展示型流程过渡到真实的模型调用链路，使 **AI** 模块真正具备可用性而不只是说明性。

第三，增强调度策略与状态机深度。未来需要在现有订单状态机和调度任务模型基础上，继续补充真实业务中的中间态、异常态和回滚逻辑，并引入更细化的优先级控制、任务调度约束和设备反馈处理机制。这样才能让 **VIP** 加急、临时取物、预调度和新能源腾桩等特色功能真正形成工程上的完整闭环。

第四，完善实时通信与数字孪生同步机制。下一阶段可逐步引入 **WebSocket** 或其他推送方式，把 **AGV** 坐标、车位状态、事件流和安全告警从当前演示式刷新，升级为更接近真实设备和后端状态变化的实时同步机制，从而进一步提升数字孪生页面的可信度和系统整体的运行感。

第五，补充系统测试和部署验证。后续应在功能联调完成的基础上，继续开展压力测试、异常流程测试、安全告警测试和接口健壮性测试，并逐步完善部署方案、环境配置和运行脚本，使项目在答辩之外，也具备更好的展示复现性和工程交付性。

总体而言，ParkVision 后续开发的主线应当从“系统实现阶段的可展示闭环”进一步升级为“面向真实场景的可验证闭环”。只要能够围绕现有工程骨架持续补齐数据底座、AI 联调、调度深度和实时链路，本项目完全具备继续向更高完成度推进的基础。